Adaptive Task Offloading and Resource Management for Vehicular Edge Computing

Li Jin, Shuaili Bao, Guoan Zhang, *Senior Member, IEEE*, Hao Zhu, Wei Duan, Qiang Sun, *Member, IEEE*, Jiayi Zhang, *Senior Member, IEEE*, and Pin-Han Ho, *Fellow, IEEE*

***Abstract*—Within the field of dynamic vehicular networks, also known as vehicular edge computing (VEC), task offloading and resource allocation face significant challenges due to constantly changing network conditions and limited resources. To address these issues, we propose a novel method for task offloading and resource allocation that considers the unpredictable starting points of vehicle offloading and the immediate impact of vehicle movement on wireless communication links. We utilizes dynamic channel modeling to track the fluctuations in link state, incorporating a task classification and migration strategy. This approach allows us to adaptively modify both single and cross-region offloading. Furthermore, to minimize overall latency and energy consumption, we optimally coordinate communication and computing resources by dynamically assigning vehicle transmission power, local computing resources, and the computing resources of roadside units. Simulation results indicate that the proposed approach effectively minimizes both latency and energy consumption. It notably outperforms benchmark schemes, showing a significant reduction in the Z value, achieving lower task delays and reduced energy consumption compared to existing strategies. These findings highlight the substantial advantages of the proposed approach, particularly in dynamic vehicular environments.**

***Index Terms*—Task offloading, resource allocation, dynamic channel modeling, task classification, vehicular edge computing.**

Received 9 April 2025; revised 5 July 2025; accepted 30 July 2025. Date of publication 11 August 2025; date of current version 19 January 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62471258, in part by the Key Research and Development Program of Nantong under Grant GZ2024002, and in part by the Nantong Municipal Science and Technology Program for Social Livelihood under Grant MS2024068. The review of this article was coordinated by Prof. Jiawen Kang. *(Corresponding author: Hao Zhu.)*

Li Jin, Guoan Zhang, and Wei Duan are with the School of Information Science and Technology, Nantong University, Nantong 226019, China (e-mail: jin.li@ntu.edu.cn; gzhang@ntu.edu.cn; sinder@ntu.edu.cn).

Shuaili Bao and Hao Zhu are with the School of Artificial Intelligence and Computer Science, Nantong University, Nantong 226019, China (e-mail: 2330320052@stmail.ntu.edu.cn; searain@ntu.edu.cn).

Qiang Sun is with the School of Information Science and Technology, Nantong University, Nantong 226019, China, and also with the Nantong Research Institute for Advanced Communication Technologies, Nantong 226019, China (e-mail: sunqiang@ntu.edu.cn).

Jiayi Zhang is with the School of Electronic and Information Engineering, Beijing Jiaotong University, Beijing 100044, China (e-mail: jiayizhang@bjtu.edu.cn).

Pin-Han Ho is with the Shenzhen Institute for Advanced Study, University of Electronic Science and Technology of China, Shenzhen 518000, China, and also with the Department of Electrical and Computer Engineering, University of Waterloo, Waterloo, ON N2L 3G1, Canada (e-mail: pinhanho71@gmail.com).

Digital Object Identifier 10.1109/TVT.2025.3597459

I. Introduction

WITHIN the transportation sector, the swift evolution of digitalization and information technology is persistently propelling the rapid advancement of intelligent transportation systems and autonomous driving technology [1]. Throughout this progression, a plethora of innovative service functions has surfaced, which results in a growing demand for computational task processing for vehicles, including real-time navigation, traffic monitoring, and multimedia content processing [2]. These tasks often require significant computational power and low latency, exceeding the capabilities of computing resources. Vehicular Edge Computing (VEC), through offloading computational tasks to edge devices, effectively alleviates the pressure on onboard resources. Although current communication methods primarily based on fifth-generation (5G) networks and cellular vehicle-to-everything (C-V2X) technologies offer substantial improvements compared to traditional approaches like dedicated short-range communications (DSRC), they remain insufficient to cope with the more complex and demanding scenarios anticipated in the future. Consequently, the emergence of sixth-generation (6G) wireless communication technology becomes inevitable. The 6G network exhibits significant advancements in data transmission rates, ultra-low latency, high reliability, and extensive network coverage, empowering VEC to achieve unprecedented enhancements in task offloading efficiency, data processing capabilities, and real-time responsiveness. This advancement will thus more effectively drive the progress of autonomous driving and intelligent transportation systems. Vehicle-to-everything (V2X) communication and vehicular edge computing (VEC) integration significantly improves vehicle communication performance by reducing latency and energy consumption, enhancing computational power, and increasing system reliability. This approach leverages the strengths of on-board and edge computing, enabling vehicles to offload tasks to nearby edge devices, thus easing the computational load on the vehicle and offering more efficient, flexible, and stable support.

In the context of the Internet of Vehicles (IoV), selecting an appropriate task offloading strategy is critical to optimizing overall system performance. Various offloading strategies have been proposed, each with its strengths tailored to different application scenarios and requirements. The core of task offloading lies in formulating the offloading strategy and allocating resources efficiently. To this end, researchers have proposed numerous

0018-9545 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Goa University. Downloaded on October 26,2026 at 08:20:24 UTC from IEEE Xplore. Restrictions apply.

solutions from different perspectives, contributing significantly to the field [4], [5], [6], [7], [8], [9], [10].

A. Related Work

Recently, with the continuous advancements in technology and methodologies, the field of task offloading has undergone profound transformations. For instance, [11] proposed a task pre-allocation and adaptive offloading strategy to optimize task offloading delays and enhance system performance. Similarly, [12] introduced a Lyapunov optimization-based task offloading method aimed at dynamically adjusting offloading decisions to optimize energy consumption and offloading efficiency in vehicular platoons. However, these methods often fail to adequately address the dynamic nature of vehicular networks, which can result in resource wastage or increased task processing delays. To tackle these challenges, collaborative edge computing offloading strategies have emerged, leveraging both on-board computational resources and edge computing resources. For example, [13] proposed a reinforcement learning-based resource allocation strategy by combining edge computing with content caching in vehicular cloud-edge computing systems. In [14], a genetic algorithm-based collaborative offloading strategy was proposed to optimize task distribution and pricing among vehicles, edge servers, and cloud resources, effectively reducing offloading delays and costs. Furthermore, [15] developed a collaborative offloading strategy, which optimized task offloading and resource allocation, balancing computation accuracy, energy consumption, and latency to improve the overall performance of VEC systems.

The aforementioned studies show that collaborative offloading strategies decrease vehicles' computational load, optimize resource usage, and enhance system performance. Dynamic offloading strategies, considering vehicle mobility and real-time network conditions, are a key research area. These strategies decide task offloading locations dynamically by predicting and adjusting offloading nodes based on network conditions and task requirements. Research methods include model-based predictions, reinforcement learning, and game theory. For example, [16] proposed a scheme based on Lyapunov optimization theory dynamically adjusts task offloading and communication resource allocation to maximize energy efficiency while meeting latency constraints, adapting to the dynamic vehicular network environment. [17] introduced a scheme optimized offloading success rates and delays by incorporating future vehicle trajectories, resource loads, and offloading delays, addressing limitations of solutions that ignore vehicle mobility.

With the rapid advancement of artificial intelligence, increasingly sophisticated algorithms, such as deep reinforcement learning, have been introduced to optimize resource allocation in IoV. These algorithms adaptively adjust offloading strategies based on real-time network states and task requirements, ensuring real-time performance while optimizing overall system efficiency. For instance, [18] proposed a deep reinforcement learning-based shared offloading strategy that adaptively selects among local execution, task offloading, and shared offloading modes to optimize task offloading delays and energy consumption in IoV. Additionally, [19] optimized task decomposition, offloading service layout, and in-vehicle computational resources to improve edge offloading success rates, reduce task offloading delays, and enhance system resource utilization. Similarly, [20] introduced a reinforcement learning-based task offloading method to optimize energy consumption and user experience by reducing base station power consumption and overdue task rates in vehicular networks. Moreover, [21] proposed a federated learning and deep learning-based task offloading method, which optimizes computation, communication, and resource utilization in vehicular fog computing (VFC) networks, achieving a balance between latency and energy consumption as well as significantly improving task offloading performance. Furthermore, [22] presented a low-complexity task offloading and resource allocation algorithm by decoupling variables in non-convex problems, leveraging a distributed optimization and federated learning framework.

Task offloading and resource allocation are crucial for efficient VEC, with node selection being key to enhancing performance in dynamic IoV environments. Current methods, focusing on single-region and multi-node optimization, often overlook the randomness of offloading locations and potential network issues, restricting their use in dynamic IoV settings.

In VEC scenarios, due to the limited communication coverage of roadside units (RSUs), vehicles traveling at high speeds inevitably experience frequent handovers [23], [24]. During these handovers, data uploads and processing may face interruptions or delays, negatively impacting task timeliness and computational efficiency. This dynamic variability imposes greater demands on task offloading strategies, particularly when task uploads span multiple coverage areas of RSUs, requiring phased task transmission. However, network instability during handovers can result in task failures or delays [25], further increasing the complexity and uncertainty of task completion. While existing studies have addressed task migration and resource allocation, few have fully explored the issue of RSU segmentation and its impact on task offloading. For instance, [26] proposed a distributed vehicular-to-road collaborative task offloading scheme with task migration mechanisms, which leveraged multi-agent deep reinforcement learning to optimize offloading targets and the number of sub-tasks, thus significantly improving task processing speed and service reliability. Similarly, [27] presented a multi-hop task offloading decision model for MEC-enabled IoV, which considered regions with insufficient edge server coverage and nearby vehicle computational resources to optimize efficiency through multi-hop transmission and collaborative computing. Furthermore, [28] addressed the challenges of incomplete task migration due to high-speed vehicle mobility and RSU overload during peak traffic periods by proposing an integrated task migration and resource allocation scheme.

However, while these methods offer solutions for task migration, they often overlook the dynamic nature of high-speed vehicle mobility [29], cross-RSU task migration, and the need for precise modeling of wireless communication link changes. In particular, the segmentation of RSU coverage areas and the coordination between multiple RSUs during task offloading remain

Authorized licensed use limited to: Goa University. Downloaded on October 26,2026 at 08:20:24 UTC from IEEE Xplore. Restrictions apply.

underexplored. RSU segmentation, which involves managing task offloading across overlapping coverage areas, is crucial for ensuring seamless handovers and minimizing delays, especially in dynamic VEC scenarios.

B. Contributions and Organization

To tackle aforementioned issues, in this paper we novelly exploit a task offloading and resource allocation strategy for dynamic vehicular network scenarios, which fully considers the randomness of vehicle offloading tasks and the real-time changes caused by vehicle mobility in wireless communication link states due to vehicle mobility. Through dynamic channel modeling, task classification and migration strategies, and joint communication and computing resource optimization, we achieve an efficient task offloading and resource optimization dynamic task offloading strategies. The main contributions of our work can be summarized as follows:

- We propose an innovative dynamic task offloading and resource allocation strategy tailored for highly dynamic vehicular networks. The entire system is modeled as a multi-objective optimization problem that jointly optimizes vehicle transmission power, local computing capabilities, and RSU computing resources. Crucially, we consider atomic tasks, which are indivisible and must be fully executed either locally or remotely, and explicitly model cross-timeslot migration, where task uploads span multiple RSU coverage durations due to high-speed mobility. This feature is often neglected in prior work, which assumes either static task placement or divisible workloads.
- To explore the feasibility of addressing the randomness of task arrivals, we exploit a novel dynamic task classification and migration strategy based on the comparison between task upload time and vehicle dwell time within RSU coverage. Tasks are categorized as single-region or cross-region offloading tasks. This enables real-time migration decisions across RSUs while preserving the atomicity of tasks. Unlike existing methods that either rely on fixed rules or ignore RSU handovers, our approach ensures consistent task execution during high-speed movement and improves system robustness.
- This study also considers a dynamic channel modeling method, which accurately captures the dynamic changes in wireless communication links caused by vehicle mobility. The proposed comprehensively considers the effects of large-scale path loss, small-scale Rayleigh fading, and Doppler shift, aiming to achieve real-time updates of channel states, as well as improve the accuracy of offloading decisions.
- Finally, to verify the adaptability of this method in high-dynamic vehicular network scenarios, we provide a comprehensive performance evaluation for different RSU deployment scenarios and vehicle dynamic characteristics. Experimental results show that the proposed strategy performs excellently in terms of performance.

The remainder of the paper is organized as follows. Section II details the application model of vehicular edge computing within traffic scenarios, concentrating on the communication and computing models crafted for vehicular tasks. It builds on this by establishing an optimization problem with the goal of reducing the weighted sum of task latency and energy usage. Section III then divides the optimization problem into two sub-problems: task classification and resource allocation. It proposes a novel hybrid optimization algorithm to decrease the total cost of vehicular task execution. Section IV shows the numerical analysis results through comparative analysis. Finally, we conclude the paper in Section V.

TABLE I
DESCRIPTION OF SYMBOLS

Symbol	Description
L_i	Load of task i
D_i	Data size of task i
T_i	Maximum acceptable delay for task i
w_i	Coordinate of vehicle i
q_i	Coordinate of the RSU j
d_{ij}	Distance between vehicle i and RSU j
x_{ij}	Task offloading decision
p_{ij}	Transmission power
R_{ij}	Uplink transmission rate
T_i^l	Local computing time
E_i^l	Local energy consumption
T_i^{tr}	Edge transmission time
T_i^0	Edge computing time
T_i^j	Total edge processing time
E_i^j	Edge energy consumption
T_i^h	Context transmission delay
α^T	Trade-off coefficients for delay
α^E	Trade-off coefficients for energy consumption
σ^2	Gaussian white noise
Z_i	The comprehensive cost of task
f_i	Local computing resource
f_i^j	Edge computing resource
f_d	Doppler frequency shift

II. SYSTEM MODEL

In this section, we first consider a VEC network architecture and then introduce the task and computational models, detailing the task classification methods [30]. It also proposes and constructs the problem of vehicle task edge placement and resource allocation. To help readers better understand, the related symbols involved in this section have been listed in Table I.

A. Traffic Scenario Model

As shown in Fig. 1, we consider a city road scenario where RSUs are placed at equal distances on the city road. Vehicles are traveling on the road at the speed of v. The road is modeled as a multi-lane straight highway with fixed lane assignments. Each vehicle is initially assigned to a specific lane, which remains unchanged throughout the simulation. The movement of vehicles is strictly along the horizontal direction, and their positions

Authorized licensed use limited to: Goa University. Downloaded on October 26,2026 at 08:20:24 UTC from IEEE Xplore. Restrictions apply.



Fig. 1. Traffic scenario of vehicular edge computing.

are updated at each time step based on a constant velocity v. The lane position is fixed, and no lane changes or overtaking behaviors are considered. The set of RSU indices is denoted by $Q = \{1, 2, \ldots, j\}$, while the set of vehicles is defined as $I = \{1, 2, \ldots, i\}$. The index of an RSU is represented by $j \in \mathbb{Q}$, and the index of a vehicle task is represented by $i \in \mathbb{I}$. The edge network consists of RSUs, and each RSU is equipped with an edge server that is capable of providing wireless access and computing resources support for vehicles, in order to process computationally intensive or latency-sensitive tasks [2]. RSUs are deployed at equal intervals along the roadside, with their positions having coordinates of q_j. Each RSU has a coverage range of *r*. To precisely describe the spatial position of the RSU and the motion dynamics of vehicles, a three-dimensional Euclidean coordinate system is employed for modeling. The height of the base station antenna, denoted as H, represents the installation height of the RSU above the ground. The x-coordinate of a roadside RSU is represented as x_j, and thus the coordinate q_j, is expressed as $(x_j, 0, H)$. The straight-line perpendicular distance from a vehicle to the roadside is denoted as e_i, and the initial coordinates of a vehicle traveling along the road, w_i, is expressed as $(x_i, e_i, 0)$. The real distance d_{ij} between the vehicle and its corresponding connected RSU is defined as follows:

$$d_{ij}(t) = \sqrt{e^2 + (w_i(t) - q_j)^2 + H^2}. \tag{1}$$

In this study, a vehicle's task can only be received if the vehicle is within the communication range of RSU j, which is modeled by

$$\|q_j - w_i(t)\| \leq r. \tag{2}$$

B. Task Offloading Model

Each vehicle has a computationally intensive or delay-sensitive task i to be executed, represented by a triplet $\langle L_i, D_i, T_i \rangle$, where L_i is the load of task i, D_i is the data size of task i, and T_i is the maximum acceptable delay for task i, indicating that the execution time of the vehicle task must not exceed the maximum acceptable delay. In this study, tasks are treated as atomic units that cannot be divided. As a result, a task must be fully executed on a single computing node and cannot be partially processed across multiple nodes. The offloading decision for tasks is represented by x_{ij}, where $x_{ij} \in \{0, 1\}$. If $x_{ij} = 0$, it indicates that the vehicle's task is executed locally. Conversely, if $x_{ij} = 1$, it shows that the vehicle's task is offloaded to the edge server for execution, which is given as

$$x_{ij} = \begin{cases} 0, & \text{local computing} \\ 1, & \text{if vehicle-i offloads task to RSU j} \end{cases} \tag{3}$$

Assuming that the vehicle task selection is performed on the edge, based on the vehicle's initial offloading location [2], moving speed, and the coverage range of the RSU, the task offloading mode can be divided into two types, which are single-region offloading and cross-region offloading.

Single-region offloading: As shown in Fig. 2, if a vehicle's estimated task transmission time T_i^{tr} is less than or equal to its remaining dwell time T_i^{zj} within the coverage of RSU_j, then $x_{ij} = 1$ is feasible under a single-region offloading model. The task is entirely transmitted to and processed by RSU_j's edge server. This method eliminates complex operations like context migration and cross-region handovers, thereby reducing communication latency and energy consumption. Channel dynamics affecting T_i^{tr} are captured through a mobility-aware model that considers vehicle speed, RSU coverage range, and

Authorized licensed use limited to: Goa University. Downloaded on October 26,2026 at 08:20:24 UTC from IEEE Xplore. Restrictions apply.

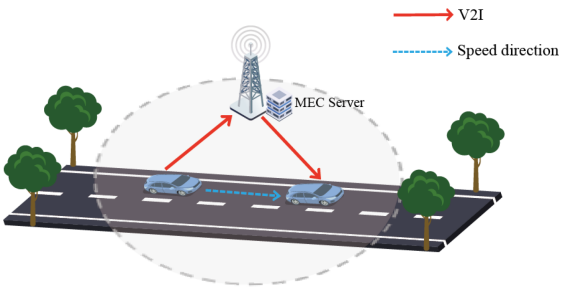


Fig. 2. Single-region offloading model.

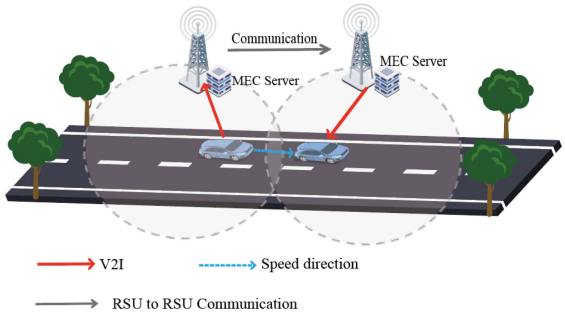


Fig. 3. Cross-region offloading model.

wireless channel conditions. This mode is suitable for tasks that are sensitive to delay or require moderate data size computation.

Cross-region offloading: When $T_i^{\text{tr}} > T_i^{zj}$, the task transmission cannot be completed within a single RSU's coverage, a cross-region offloading strategy is employed as the Fig. 3 shows. In this case, $x_{ij} = 1$ still holds, but the task data is transmitted in segments across adjacent RSUs as the vehicle moves. The partially transmitted data from RSU_j is forwarded to the next RSU_{j+1} through high-speed links. This approach supports tasks with large data volumes and high computation requirements, though it incurs higher communication and coordination overhead. Nevertheless, it maintains task atomicity by ensuring all data reaches the final execution node before processing begins. The cooperative behavior among RSUs is key to preserving service continuity and latency bounds under mobility constraints.

By choosing the appropriate offloading decision [33], the balance between energy and delay for vehicle tasks can be achieved. In this study, tasks are considered as atomic tasks that can't be divided. Therefore, tasks cannot be partially computed on different computing nodes.

C. Communication Model

This paper proposes a comprehensive channel model combining Doppler shift, path loss, and small-scale fading to accurately depict channel characteristics. The model captures the dynamic channel variations over time in urban road environments and incorporates the Doppler shift effects in high-speed vehicular scenarios, providing a theoretical foundation for real-time, efficient communication optimization. The channel gain g_{ij} is a combination of Rayleigh fading h_{ij} and path loss PL, and its dynamic variation is mainly influenced by Doppler shift, Rayleigh fading, and changes in vehicle position. Then, the channel gain g_{ij} can be formulated as

$$g_{ij} = (PL_{ij}(t))^{-\frac{1}{2}} h_{ij}(t). \tag{4}$$

Doppler shift is the change in signal frequency caused by the relative velocity change between a vehicle and RSU, and its calculation formula is:

$$f_d(i,j) = \frac{v}{c} f_0, \tag{5}$$

where v represents the relative velocity between vehicle i and RSU j, c denotes the speed of light, commonly approximated as $c = 3 \times 10^8$ m/s, and f_0 signifies the carrier frequency of the signal. By continuously updating the Doppler frequency shift, the channel gain can be dynamically tuned. Rayleigh fading is typically modeled as a complex Gaussian random process, and its frequency domain representation is affected by Doppler frequency shift [31]. In discrete-time simulations, h_{ij} can be updated in real-time using the following formula:

$$h_{ij}(t) = h'_{ij} \cdot \exp(l_i 2\pi f_d(i,j)\, t), \tag{6}$$

where h'_{ij} is the complex coefficient of static Rayleigh fading, usually generated using a standard complex Gaussian distribution. Path loss [33] is the attenuation of a signal as it propagates through space, and its calculation formula can be written as

$$PL_{ij}(t) = 128.1 + 37.6 \cdot \log_{10}(d_{ij}(t)), \tag{7}$$

where $d_{ij}(t)$ denotes the distance between vehicle i and RSU j at time t. By updating the vehicle's position in real-time, the model achieves dynamic adjustment of path loss. Based on the Shannon model, the transmission rate of a task at one time slot is represented as

$$R_{ij}^{(t)} = B \log_2 \left(1 + \frac{p_{ij} g_{ij}^{(t)}}{\sigma^2}\right), \tag{8}$$

where B is the bandwidth of vehicles, p_{ij} indicates transmission power between vehicle i and RSU j, respectively. σ^2 represents the gaussian white noise.

D. Local Computing Model

In this case, the execution of the task is entirely dependent on the vehicle's own computational capabilities. For task i, its computational delay can be represented as

$$T_i^l = \frac{L_i}{f_i}, \tag{9}$$

where f_i represents the number of central processing unit (CPU) cycles provided by vehicle i. The local computing energy consumption stems from the power consumed by the vehicle's processor during task execution, which is closely related to the CPU operating frequency and the load of task. The local computing energy consumption model can be described by the formula as

$$E_i^l = k\,(f_i)^3\, T_i^l, \tag{10}$$

where k represents the energy consumption coefficient at the unit frequency, which reflects the inherent characteristics of

Authorized licensed use limited to: Goa University. Downloaded on October 26,2026 at 08:20:24 UTC from IEEE Xplore. Restrictions apply.

the processor's energy efficiency [32], and the specific value depends on the hardware design. Here, k is set to 2.5×10^{-28} .

E. Edge Computing Model

When the vehicle decides to offload tasks to the corresponding RSU edge servers [34] for processing, the system compares the upload time of the vehicle's dwell time within the coverage area of current RSU to determine the offloading strategy. Edge computing can be divided into two scenarios.

1) single-region offloading: When the task's upload time is less than the vehicle's dwell time at the current RSU's coverage area, the upload time depends on the task's data volume and the wireless link's transmission rate, with transmission delay being as

$$T_i^{\text{tr}} = \sum_{t=1}^{N_{\text{t}}} \frac{D_i/N_{\text{t}}}{R_{ij}^{(t)}} \tag{11}$$

where N_{t} is the number of time slots. After the edge server receives the task data, it initiates the task execution process. The execution time is expressed as

$$T_i^o = \frac{L_i}{f_i^j}, \tag{12}$$

where f_i^j represents the computational resources allocated to vehicle i by RSU j. Therefore, the total delay for a single-region offloading task i is the sum of the transmission delay and the computation delay:

$$T_i^j = T_i^{\text{tr}} + T_i^o. \tag{13}$$

Finally, the energy consumption for offloading execution of vehicle task i can be expressed as

$$E_i^j = P_{ij} T_i^{\text{tr}}. \tag{14}$$

2) cross-region offloading: When upload time exceeds vehicle's stay time in the current RSU, the tasks must be offloaded using a cross-region strategy. Vehicles partially upload data to current RSU based on stay time. The stay time is given by

$$T_i^{zj} = \frac{2jr - w_i}{v}, \tag{15}$$

where $2jr - w_i$ represents the remaining distance of vehicle i within the current RSU coverage range, and v_i represents the traveling speed of vehicle i. When vehicle i enters the coverage range of the next RSU, the remaining task data is transmitted to the current RSU, and the transmission delay can be represented as:

$$T_i^{re} = \sum_{t=1}^{N_{\text{t}}} \frac{D_{re}/N_{\text{t}}}{R_{i(j+1)}{}^{(t)}}, \tag{16}$$

where D_{re} represents the remaining task data that needs to be transmitted for vehicle task i, and $R_{i(j+1)}$ represents the wireless link transmission rate between vehicle i and the $R_{i(j+1)}$. The task data transmitted can be transferred to the RSU_j when the vehicle i enters the area through communication between $RSU_{(j+1)}$. Assuming that the context transmission delay between RSU_j and $RSU_{(j+1)}$ is known and is proportional to the task computation workload, the estimated context transmission delay for task i is:

$$T_i^h = L_i \Gamma, \tag{17}$$

where $\Gamma > 0$ [35], upon completion of the task data upload, the edge server processes vehicle task i, and the execution delay of task *i* is represented as:

$$T_i^c = \frac{L_i}{f_i^j}. \tag{18}$$

At this point, the total delay of the task including the maximum value of the vehicle task's dwell time at the previous RSU, the context transmission delay, and the transmission time of the vehicle task at the next RSU, plus the execution delay of the task can be expressed as

$$T_i^j = T_i^{zj} + \max(T_i^{re}, T_h) + T_i^c. \tag{19}$$

The total energy consumption of the task including the energy consumption of two communication segments of the task can be given by

$$E_i^j = P_{ij} T_i^{zj} + P_{ij} T_i^{re}, \tag{20}$$

F. Problem Formulation

To optimize the performance of task processing, we design an integrated cost model. This model not only deeply considers key factors such as time delay and energy consumption but also integrates other variables that may affect task execution efficiency and resource consumption, ensuring optimal efficiency and the lowest resource consumption when executing tasks.

The comprehensive cost for task processing, denoted as Z_i, is defined as a weighted sum of time delay and energy consumption and is represented as:

$$Z_i = (1 - x_{ij}) Z_i^l + x_{ij} Z_i^j, \tag{21}$$

Z_i^l denotes the comprehensive cost of local processing, which is formulated as $\alpha^T T_i^l + \alpha^E E_i^l$. Meanwhile, $\alpha^T T_i^j + \alpha^E E_i^j$ describes the comprehensive cost of edge processing, denoted as Z_i^j. Where α^T and α^E are the trade-off coefficients for delay and energy consumption [33], used to balance the weights of time delay and energy consumption in the target optimization.

This study examines random initial vehicle positions during offloading, co-optimizing task placement and resource allocation within constraints of task delays and computational resources. The goal is to determine the optimal computation task offloading strategy to minimize total vehicle task execution cost, represented by the weighted sum of task delay and energy consumption. The problem is formulated as follows:

$$\min \sum_{i=1}^{I} z_i$$

$$\text{s.t.} \quad \text{C1: } x_{ij} \in \{0, 1\},$$

$$\text{C2: } (1 - x_{ij}) T_i^t + x_{ij} T_i^j \leq T_i,$$

Authorized licensed use limited to: Goa University. Downloaded on October 26,2026 at 08:20:24 UTC from IEEE Xplore. Restrictions apply.

$$\begin{aligned}
&\text{C3: } \sum_{j=1} x_{ij} = 1,\\
&\text{C4: } x_{ij}\|q_j - w_i(t)\| \le r_j,\\
&\text{C5: } \sum_{i\in N_i} x_{ij} f_i^j \le f_j,\\
&\text{C6: } 0 < f_i \le f_{i,\max},\\
&\text{C7: } 0 < P_i < P_{\max}.
\end{aligned} \tag{22}$$

Note that C1 denotes a binary decision pertaining to task offloading, signifying that tasks must be executed either locally or offloaded to the pertinent edge server. C2 mandates that the completion timeframe for vehicle-related tasks does not surpass their respective maximum tolerable delay T_i. C3 imposes a constraint whereby each vehicle task is restricted to selecting a single RSU_j for offloading purposes. C4 guarantees that the vehicle's geographical position lies within the coverage area of the associated RSU. C5 stipulates that the aggregate task resources offloaded onto the corresponding edge server must not exceed the server's resource capacity limit. C6 indicates that the computing resources allocated locally to the vehicle must not transcend its maximum allowable computing resource threshold. C7 stipulates that the transmission power allocated to the vehicle must remain within the prescribed maximum transmission power limitation.

III. Problem Solution

This section aims to minimize comprehensive task costs. It addresses a mixed integer nonlinear programming (MINLP) optimization problem, involving both integer and continuous variables [36], and nonlinear feasible regions. We introduce a heuristic algorithm based on dynamic rules and logic to optimize task classification and connection strategies, considering differences between single-region task offloading and cross-region task collaboration. The algorithm gradually optimizes task offloading decisions and resource scheduling. For resource allocation, we use the Lagrangian relaxation method to determine the optimal resource distribution across RSUs and edge servers [37].

A. Task Classification Algorithm

The proposed task classification and offloading decision process is inherently modular, and each vehicle can make its own decision based on local parameters such as estimated dwell time and instantaneous channel state information. This modularity enables the possibility of distributed execution.

Under constraint C1, vehicle tasks must be executed locally or offloaded to edge servers. If task i is offloaded, the MEC server obtains vehicle status, including location and speed, to determine if it's within the current RSU's coverage and establish a preliminary connection. If vehicle i is in the overlapping coverage of multiple RSUs, the optimal one is selected based on location and channel state. The preliminary connection matrix optimizes transmission delay of task i and calculates driving time within the communication range. If driving time T_i^{tr} is less than transmission delay T_{zj}, task i is processed by the connected RSU's server; otherwise, cross-RSU collaborative computation is required.

Algorithm 1: Task Classification Algorithm.

Input: $P_{\max}, f_j, D_i, L_i, T_{\max}, g_{ij}, B$
Output: P_{ij}, f_{ij}
Initialize P_{ij} and f_i^j randomly for connected RSUs.
for $t = 1$ **to** max_iterations **do**
  Update channel gains g_{ij} using path loss and small-scale fading.
  Assign vehicles to RSUs based on the highest g_{ij} if $d_{ij} < r$.
  Optimize edge offloading resources:
  Update P_{ij} using Newton iteration with constraints $C7$.
  Compute execution time T_{exe} and communication delay T_{tr}
  Update λ_i using gradient adjustment.
  Compute total delay $T_{\text{total}} = T_{\text{tr}} + T_{\text{exe}}$.
  Evaluate RSU capacity and adjust load balancing:
  Update μ_i using gradient adjustment.
  Update single and multi-unit offloading decisions:
  if $T_{\text{tran}} \le T_{\text{stay}}$ **then**
    Mark as single-unit offloading.
  else
    Prepare for multi-unit offloading and adjust connection matrix.
  end if
end for
return Connection matrix and resource allocation P_{ij}.

In the case of overlapping RSU coverage, where a vehicle is simultaneously within communication range of multiple RSUs, we define a decision criterion to determine the offloading destination. The vehicle evaluates the real-time channel gain with each candidate RSU, considers the remaining task data to be uploaded, and accounts for the estimated dwell time left within the current RSU. If the predicted completion time of the task upload exceeds the dwell time, the system triggers a partial upload followed by inter-RSU migration. The remaining data is forwarded to the next RSU, which continues task reception and execution. Otherwise, the task is entirely offloaded to the current RSU to minimize migration overhead.

This study proposes a heuristic algorithm given in Algorithm 1 that uses dynamic rules and logic to optimize task classification and connection strategies. It differentiates between single-region task offloading and cross-region task collaboration, progressively improving offloading decisions and resource scheduling. The algorithm quickly determines if tasks can be handled by a single RSU or need multiple RSUs' collaboration by comparing vehicle upload times with dwell times, establishing a dynamic task decision mechanism for high-speed vehicle scenarios. In resource allocation optimization, the algorithm employs a multi-level dynamic adjustment strategy that channel gain guides vehicle-RSU connection switching, Newton's iterative method optimizes task transmission power, and computing

Authorized licensed use limited to: Goa University. Downloaded on October 26,2026 at 08:20:24 UTC from IEEE Xplore. Restrictions apply.

resource allocation is dynamically adjusted based on task execution delay to maximize resource utilization efficiency under limited resources. To address vehicle movement dynamics, the algorithm incorporates a dynamic connection matrix update mechanism, ensuring seamless transitions during multi-unit offloading and efficient collaboration among RSUs. The strategy, combining dynamic channel modeling with heuristic resource optimization, significantly improves edge computing resource management efficiency, especially in high-dynamic scenarios. The algorithm's practical value and implementation potential are substantial, with detailed pseudocode outlining specific implementation steps.

B. Resource Allocation Algorithm

The resource allocation framework as summarized in Algorithm 2 is derived from a Lagrangian dual decomposition, which allows the global optimization problem to be decoupled into local subproblems at vehicles and RSUs. Each RSU allocates resources to associated vehicles based on local demands and Lagrange multipliers, enabling distributed implementation with coordination via dual variables.

Objective function is weighted sum of task delays and energy consumption across all vehicles, reflecting their autonomy in task execution. Optimization objectives are linked due to resource constraints. Total resource demands of vehicles at same RSU must be considered. Constraints C5 and C6 ensure cumulative computational resource requirements of vehicles connected to an RSU do not exceed its capacity, fostering vehicle interdependence. Resource coupling complicates isolated optimization of task allocation for each vehicle. Lagrange multipliers λ_i and μ_i are introduced for task delay and computational resource constraints, respectively. Lagrange function is represented as:

$$\begin{aligned} L(\lambda,\mu,X,P,F) &= \sum_i \left[(1-x_{ij})Z_i^l + x_{i,j}Z_i^j\right] \\ &+ \sum_i \lambda_i \left[(1-x_{ij})T_i^l + x_{ij}T_i^j - T_{i,\max}\right] \\ &+ \mu_i \left(\sum_{i\in N_i} x_{ij}f_i^j - f_j\right), \end{aligned} \tag{23}$$

where, N_i represents all vehicles connected to RSU_j. By introducing the Lagrangian function L, the dual problem is defined as $g(\lambda,\mu)$, which represents the minimum value achievable by the original problem under the constraints of the delay multiplier λ and the computation resource multiplier μ. The dual problem is represented as:

$$g(\lambda,\mu) = \inf_{X,P,F} L(\lambda,\mu,X,P,F). \tag{24}$$

Adopting alternating optimization, the dual problem is split into two layers of sub-problems: the inner one involves vehicle task offloading decision-making x_{ij}. The problem can be formulated as

$$x_{ij}^* = \arg\min_{x_{ij}} \left\{ Z_i^l \cdot (1-x_{ij}) + Z_i^j \cdot x_{ij} \right\}. \tag{25}$$

For local computing, the problem focuses on optimizing local resource allocation. We adopt a distributed approach to allocate resources and determine the optimal CPU frequency for vehicle tasks. The key challenge is minimizing the comprehensive cost by identifying optimal computing resources, i.e.,

$$\begin{aligned} \min_{f_t} \ & \left(\alpha^T + \lambda_i\right)\frac{L_i}{f_i} + \alpha^E k (f_i)^2 L_i \\ & \text{s.t. C6.} \end{aligned} \tag{26}$$

Given the convex nature of the objective function, when $0 < f_i < f_{i,\max}$, the function exhibits convexity. we can apply the Karush-Kuhn-Tucker (KKT) conditions to derive the optimal solution as

$$f_i^* = \max\left(0, \min\left(\sqrt[3]{\frac{\alpha^T+\lambda_i}{2\alpha^E k}}, f_{i,\max}\right)\right). \tag{27}$$

In the execution at the edge of tasks, achieving comprehensive cost minimization requires the joint optimized allocation of communication and computational resources, applicable to both single-region and cross-region offloading scenarios. *1) Communication resources:* The determination of the optimal communication resources mainly relies on the optimization of transmission power. It is necessary to solve a specific optimization problem as follows:

$$\begin{aligned} \min_{P_t} \ & \left(\alpha^T + \alpha^E P_{ij} + \lambda_i\right)\frac{D_i}{R_{ij}} \\ & \text{s.t. C7.} \end{aligned} \tag{28}$$

The objective function, involving logarithmic and fractional components,is highly nonlinear and difficult to solve analytically. Therefore, we compute the first and second derivatives and employ Newton's method for iterative optimization, with the update rule

$$P_{ij}^{(k+1)} = P_{ij}^{(k)} - \frac{f(P_{ij}^{(k)})}{f'(P_{ij}^{(k)})}. \tag{29}$$

Given convex characteristics of the objective function, Newton's method, leveraging function and derivative information, can efficiently find the global optimal solution.

2) Computational resources: The problem of solving for optimal edge computing resources can be reformulated as

$$\begin{aligned} \min_{f_t} \ & \left(\alpha^T + \lambda_i\right)\frac{L_i}{f_t} + \mu_i \sum_{i\in N_t} f_i^j \\ & \text{s.t. C5.} \end{aligned} \tag{30}$$

The function exhibits convexity with respect to f_i^j, thus making the KKT conditions key determine the optimal solution. By taking the first derivative with respect to f_i^j and applying the KKT conditions, the optimal allocation of computing resources is obtained as

$$f_i^{j*} = \max\left(0, \min\left(\sqrt{\frac{(\alpha^T+\lambda_i)L_i}{\mu_i}}, f_j\right)\right). \tag{31}$$

Authorized licensed use limited to: Goa University. Downloaded on October 26,2026 at 08:20:24 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: Resource Allocation with Task Offloading.

Input: Initialize parameters $\lambda_i, \lambda_l, \mu_i, P_{ij}, f_{ij}$.
Output: $Z_i, T_i^l, T_i^j, E_i^l, E_i^j$

```
Calculate x_ij connection matrix using Algorithm 1.
for iter_l = 1 to outer_iterations do
  for iter = 1 to max_iterations do
    for each vehicle i do
      Compute local CPU frequency f_i by (29).
      Offloading Task Optimization:
      if multi_unit_unload(i) = 1 then
        Calculate remaining data D_se(i).
        Update path loss, channel gain, and
        transmission power P_ij by (31).
        Optimize computation resource f_ij for
        updated RSU j by (32).
        Compute total execution delay, energy, and
        cost.
      end if
    end for
    for each vehicle i do
      Compare local comprehensive cost Z_i^l and edge
      comprehensive cost Z_i^t.
      if Z_i^l > Z_i^t then
        offload_decision(i) ← 1
      else
        offload_decision(i) ← 0
      end if
      Update connection matrix according to
      offload_decision(i) by (27).
      if offload_decision(i) = 0 then
        Remove connection to the current RSU.
        Re-optimize computation resources according
        to updated connection matrix.
        Update T_i^j, E_i^j, Z_i^j.
      end if
    end for
  end for
end for
returnFinal Z_i, T_i^l, T_i^j, E_i^l, E_i^j.
```

We determine the global computing resource distribution strategy, considering task offloading decisions and the connection matrix, to minimize the comprehensive cost Z_i. Verify if the change in Lagrange multipliers is below the threshold ϵ to meet the convergence criterion. Lagrange multipliers are updated using the subgradient method, with the update formula specified.

$$\lambda_i^{m+1} = \lambda_i^m + \delta \left(T_{i,\max} - T_i^j \right), \tag{32}$$

$$\mu_i^{m+1} = \mu_i^m + \eta \left(f_j - N_i f_i^j \right), \tag{33}$$

where δ and η represent the step size updated in each iteration.

TABLE II
VALUE OF PARAMETERS

Parameter	Value
I	[10, 30]
Q	5
T_i	[0.5 - 2] s
H	20 m
B	5 MHz
e	[5, 20] m
D_i	[0.5, 2] MB
L_i	[0.5, 3] Gcycles
p_{max}	30 dBm
f_j	[10, 50] GHz
$f_{i,max}$	[1, 5] GHz
v	[50, 80] km/h
Γ	0.05 s/Gcycles
σ^2	2×10^{-12} w

C. Computational Complexity

Algorithm 1 iterates through the vehicles and base stations to update communication states and resource allocation, with a time complexity of $O(I \times Q)$ per iteration. Algorithm 2 involves optimizing combinations of vehicles and base stations, with each optimization having a complexity of O(1), so its time complexity is also $O(I \times Q)$.

Considering both algorithms, the total time complexity is $O(I \times Q + I \times Q)$. Based on the properties of Big-O notation, the final time complexity is $O(I \times Q)$.

IV. Simulation Results

This section provides an in-depth examination of the simulation outcomes to evaluate the efficacy and efficiency of the suggested offloading strategy.

A. Parameter Setting

We consider a 1000-meter urban expressway scenario with Roadside Units deployed every 200 meters, each equipped with an edge computing server. This layout reflects typical small-cell deployments in 5G/6G vehicular networks. Vehicles move with randomly assigned speeds within the range of 50–80 km/h. Each vehicle generates computational tasks characterized by data sizes uniformly distributed in [0.5, 2.0] MB and computational workloads in [0.5, 3.0] Gcycles, which are consistent with typical workloads in autonomous driving, multimedia processing, and real-time navigation. Tasks can be executed either locally or offloaded to edge servers. Offloading is further classified into single-region and cross-region, reflecting realistic mobility-driven execution shifts. These settings are designed to capture the heterogeneous and dynamic nature of vehicular edge computing environments. The full list of simulation parameters is summarized in Table II.

To evaluate the effectiveness of the proposed scheme, this study conducted a series of experiments and set up the following plans for comparative analysis.

Authorized licensed use limited to: Goa University. Downloaded on October 26,2026 at 08:20:24 UTC from IEEE Xplore. Restrictions apply.

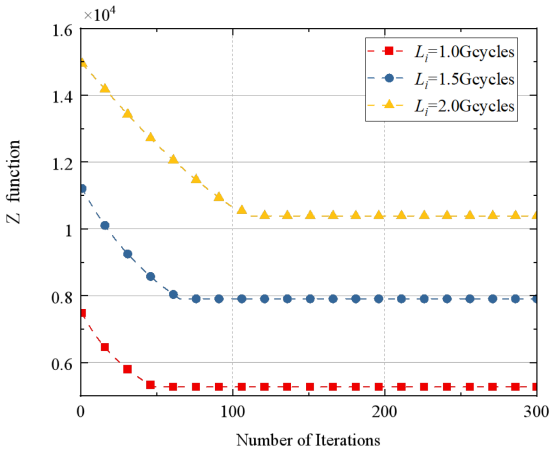


Fig. 4. Convergence analysis under different load of tasks.

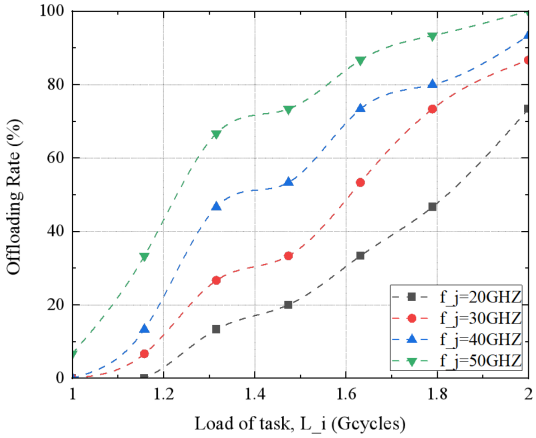


Fig. 5. Impact of the load of tasks under different computing capability of edge servers on the offloading rate.

1) Full local computing (FLO): This initiative seeks to perform all vehicular tasks directly on the vehicle, avoiding any offloading decisions.

2) Full edge offloading (FEN): In stark contrast to local computing, this approach involves offloading all tasks that require processing to edge servers, bypassing the use of onboard vehicle computing.

3) Efficient Task Offloading Scheme (ETOS): This strategy suggests that tasks should only be offloaded to MEC servers when local computing cannot satisfy the task's maximum tolerance delay requirement, thereby ensuring computational efficiency and response speed [38].

4) Position-Aware Task Offloading Strategy (PATOS): This scheme introduces a decision-making mechanism that is based on the vehicle's operational location and the deployment of RSUs [39]. During task offloading, the system allocates tasks to the closest RSU, and then each RSU is tasked with optimizing resource allocation.

5) Deep Q-Network (DQN) based Task Offloading: This approach would dynamically adjust offloading decisions by using reinforcement learning to minimize task comprehensive cost based on real-time environment feedback.

B. Convergence Analysis

Fig. 4 demonstrates convergence dynamics under heterogeneous task loads, plotting comprehensive cost Z against iteration counts. The analysis reveals convergence occurs when three criteria synchronously satisfy predefined thresholds. In the experimental setup, the number of vehicles is fixed at 30, with a maximum of 300 iterations. The algorithm's convergence behavior is evaluated under task loads of 1.0, 1.5, and 2.0 Gcycles. The results demonstrate that the overall task cost gradually decreases with an increasing number of iterations, eventually reaching a steady state. The result underscores the algorithm's robust convergence performance. In addition, it reveals that the magnitude of the task load significantly affects both the convergence rate and the final cost. Specifically, a lower task load not only accelerates the convergence rate but also reduces the final cost.

C. Offloading Rate Analysis

Fig. 5 illustrates the correlation between task offloading rate and task load L_i for various edge server computing capacities f_i^j. The graph features four curves representing edge-server capacities of 20 GHz, 30 GHz, 40 GHz, and 50 GHz. As task load increases, the offloading ratio rises, indicating vehicles offload more tasks to edge servers to alleviate local resource strain. Higher CPU frequencies result in greater offloading ratios due to superior computing power, which reduces local task execution time and transmission delays, as well as enhances offloading efficiency. As task load increases, local computing resources near their limits, further encouraging offloading to edge servers. High-frequency edge servers process tasks quickly, ensuring efficient offloading.

D. Z Function

Fig. 6 depicts the Z function defined as the comprehensive cost of this system across various traffic densities under different edge servers' computing capacities at 20 GHz, 25 GHz, and 30 GHz from left to right. Our scheme consistently yields the lowest Z-value, outperforming others in minimizing task delay and energy consumption. The DQN-based approach demonstrates competitive performance, ranking second among all compared methods due to its model-free learning capability that automatically adapts to dynamic vehicular environments without requiring explicit system modeling. However, DQN's performance remains suboptimal compared to our proposed method because the ϵ-greedy exploration strategy introduces suboptimal actions during learning phases and neural network function approximation can't perfectly capture the continuous optimization landscape. FLO suffers from high Z-values due to vehicle computing constraints, while FEN faces server overload. ETOS and PATOS show moderate performance but lack global optimization. The performance gap between DQN and our method decreases as server capacity increases, since abundant computational resources reduce optimization complexity and enable better neural network approximation accuracy.

Authorized licensed use limited to: Goa University. Downloaded on October 26,2026 at 08:20:24 UTC from IEEE Xplore. Restrictions apply.

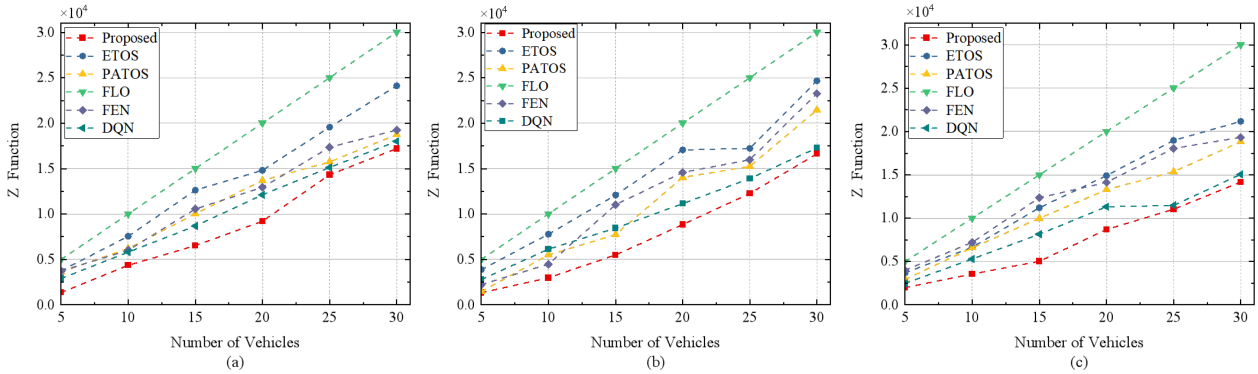


Fig. 6. Impact of the number of vehicles on the Z function (a) Edge server CPU of 20 GHz (b) Edge server CPU of 25 GHz (c) Edge server CPU of 30 GHz.

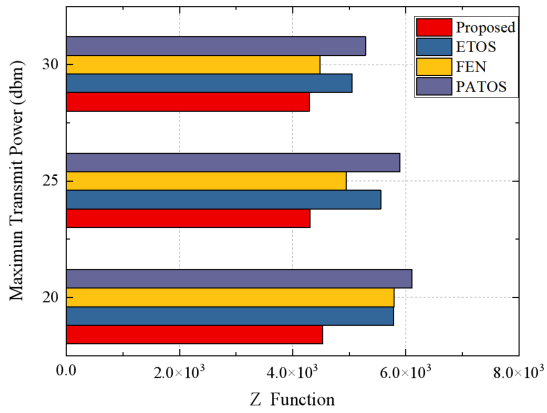


Fig. 7. Impact of the maximum transmit power on the Z function.

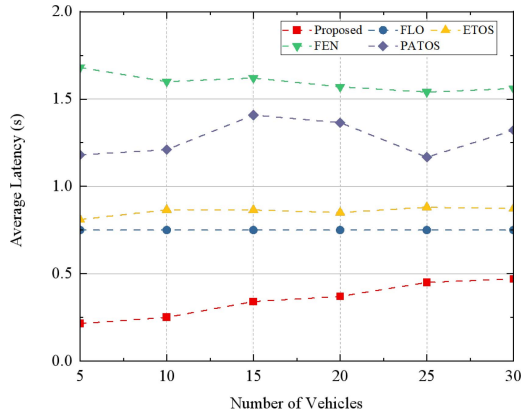


Fig. 8. Impact of the number of vehicles on the total latency.

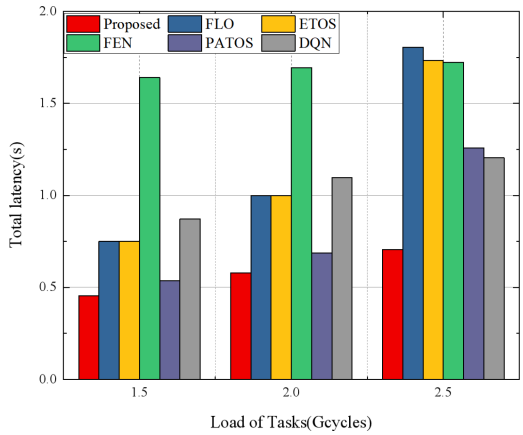


Fig. 9. Impact of the load of tasks on the total latency.

Moreover, DQN exhibits excellent scalability and maintains stable performance across varying traffic densities, demonstrating its strong adaptability for large-scale vehicular network deployments. Under different conditions of maximum transmission power, the comprehensive cost comparison of various schemes is displayed in the Fig. 7. Results indicate that higher transmission power positively affects transmission rate, as the Z value of comprehensive cost decreases with increased power. The PATOS scheme, which assigns tasks to the nearest RSU, does not ensure a stable task transmission channel, leading to increased delay and energy consumption. The ETOS scheme's performance is similar to FEN, both of which are capability of reducing task delay through offloading but not optimally adapting to various power conditions due to resource and scheduling limitations. In contrast, our scheme optimizes task transmission and execution resource configuration through global scheduling, minimizing comprehensive cost. In contrast, our scheme optimally balances edge and local computing through joint task offloading and resource allocation, minimizing transmission and execution delays. As a result, the proposed scheme consistently achieves the lowest delay across all scenarios, where the proposed scheme exhibits the most gradual delay increase, as well as highlights its superior adaptability and efficiency.

E. Latency

Fig. 8 demonstrates average delay variations under different vehicle densities, with a maximum tolerable delay set at 2 seconds. Our proposed method consistently achieves the lowest average delay across all vehicle densities, significantly outperforming other approaches. FLO maintains stable performance through consistent local computing, while ETOS shows delays around 0.85 s via selective offloading strategies. PATOS exhibits moderate delays around 1.2-1.4 s with some fluctuation due to position-based decisions, and FEN maintains the highest delays around 1.5-1.6 s, indicating resource contention when all tasks are offloaded to edge servers.

Fig. 9 illustrates total system latency under varying task loads. Our proposed method achieves minimum latency across

Authorized licensed use limited to: Goa University. Downloaded on October 26,2026 at 08:20:24 UTC from IEEE Xplore. Restrictions apply.

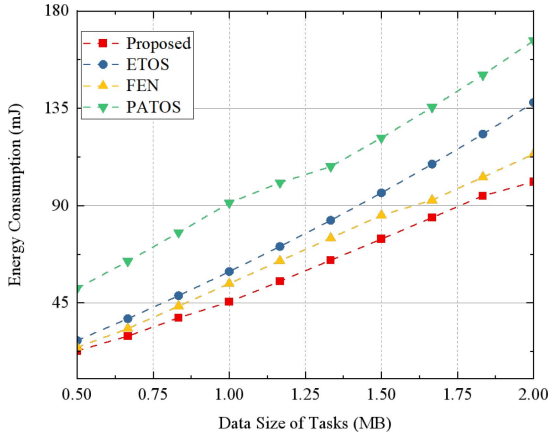


Fig. 10. Impact of the data size of tasks on the energy consumption.

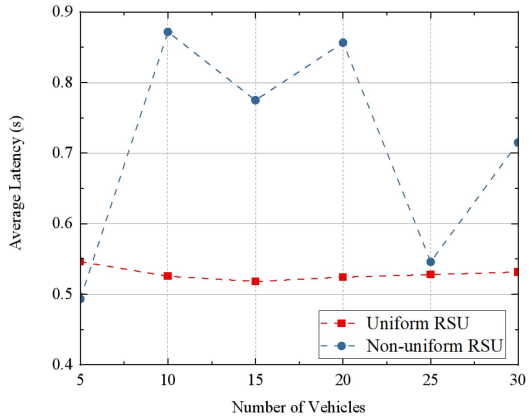


Fig. 11. Impact of the RSU distribution on the average latency.

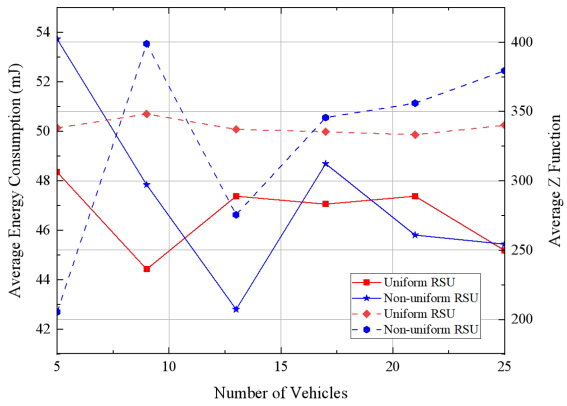


Fig. 12. Impact of the RSU distribution on the Z function and energy consumption.

all scenarios, while DQN shows moderately higher delay but still significantly outperforms traditional schemes through its intelligent decision-making mechanism that handles complex, non-linear relationships between system variables without requiring predefined rules. The performance limitation of DQN compared to our analytical optimization framework arises from temporal learning delays where the agent must accumulate sufficient experience before making optimal decisions and Q-value approximation errors that emerge in high-dimensional state-action spaces. FLO and FEN exhibit the highest latency due to computational bottlenecks and queuing delay respectively. Despite these limitations, DQN demonstrates superior adaptability over conventional rule-based methods, particularly excelling in scenarios with incomplete system information. The latency growth pattern confirms that intelligent offloading becomes increasingly critical for computation-intensive vehicular applications.

F. Energy Consumption

Fig. 10 shows that as task data increases, energy consumption rises for all strategies, though at different rates. A fair comparison of edge execution strategies with 20 simulated vehicles reveals that the proposed joint optimization of task offloading and resource allocation effectively balances transmission and computation energy consumption by dynamically scheduling tasks between local computing and edge servers. In contrast, the ETOS strategy has slightly higher energy consumption, favoring local processing for smaller task volumes but offloading tasks that exceed local delay constraints as volumes grow. The FEN strategy, which offloads all tasks to edge servers, performs well initially but experiences a sharp increase in total energy consumption due to dominant transmission energy costs as task data increases. Lastly, the PATOS strategy performs poorlywith the observation of sharply increasing energy consumption. It is because that the task assignment is based solely on vehicle proximity to RSUs, which lacks global optimization, thus causing resource overload and overlapping energy consumption for tasks.

G. RSU Distribution

Our experiments compare two RSU deployment strategies across varying vehicle densities: uniform distribution with evenly spaced RSUs, and non-uniform distribution with strategic clustering at traffic hotspots.The experimental results reveal significant differences in delay performance between the two distribution strategies. As is shown in Fig. 11, the non-uniform RSU distribution exhibits substantial delay variability, with average delays ranging from approximately 0.5 s to 0.9 s across different vehicle densities. In contrast, the uniform RSU distribution demonstrates remarkably stable delay performance, maintaining consistently low delays around 0.5 s regardless of vehicle density variations. Fig. 12 shows that both distribution strategies exhibit strikingly similar energy consumption and Z-value patterns, with average energy consumption and Z-value hovering consistently within the ranges of 40–55 and 340–400, respectively. This underscores that the selection of RSU distribution strategies exerts a more pronounced influence on delay performance than on other metrics within our algorithm.

H. Weighting Parameter

To further reveal the impact of changes with energy consumption and delay weight factors on task energy consumption and

Authorized licensed use limited to: Goa University. Downloaded on October 26,2026 at 08:20:24 UTC from IEEE Xplore. Restrictions apply.

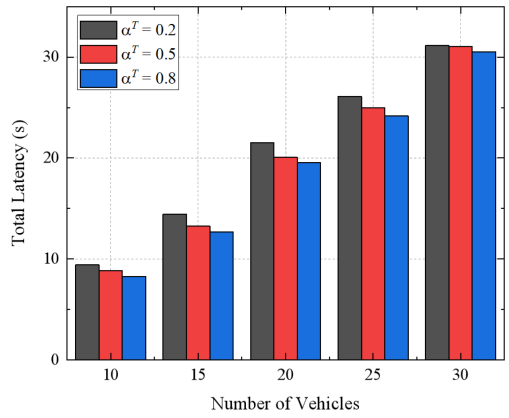


Fig. 13. Impact of the number of vehicles under different α^T on the total latency.

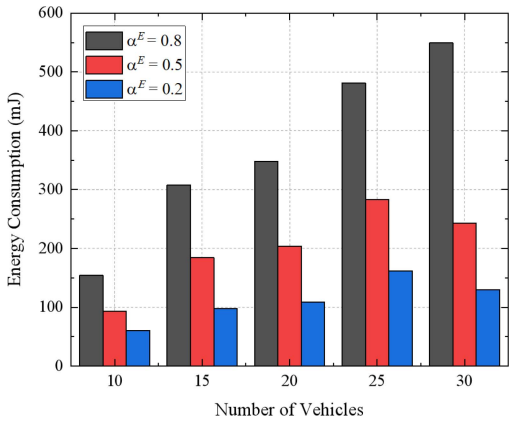


Fig. 14. Impact of the number of vehicles under different α^E on the energy consumption.

total delay, we set different trade-off factor values and consider scenarios with different traffic densities in order to observe the trends in energy consumption and total delay. The simulation results are presented in Figs. 13 and 14, respectively. The study examined the phenomenon that task delay increases with more vehicles and consequential heightened competition for computing and communication resources, which would lead to longer task completion times. Performance varies significantly with different delay weight factors α^T. A low factor prioritizes energy optimization, resulting in higher delays, while a higher factor favors reducing task delays, with minimal increase as vehicle numbers rise. This suggests that increasing the delay weight factor can enhance resource scheduling efficiency and reduce task delays, beneficial for delay-sensitive applications. Fig. 14 shows that task energy consumption rises with more vehicles, mainly due to increased energy for data transmission and computation. Energy consumption trends with weight factors mirror those of delay, with higher energy weight factors reducing task energy consumption but potentially increasing delays. Simulations in Figs. 13 and 14 demonstrate a trade-off between energy and delay optimization. Fine-tuning weight factors to balance energy and delay can effectively optimize system performance.

V. Conclusion

This paper addressed task offloading and resource allocation in dynamic vehicular networks, focused on a dynamic optimization strategy that considered the stochastic nature of tasks and vehicle movement. A dynamic channel model was developed to track real-time changes in wireless communication links. A classification and migration strategy for tasks was also presented, based on vehicle presence duration. This approach enabled a seamless combination of both single-unit and multi-unit offloading.

The strategy synergistically optimized communication and computing resources, reducing system latency and energy usage significantly. Experimental results showed significant advantages in latency and energy consumption. In future work, we aim to extend this centralized evaluation framework into a fully distributed system using multi-agent reinforcement learning or federated optimization techniques, to further enhance scalability and autonomy under realistic network conditions.

Appendix A
Proof of Convexity

The problem aims to minimize the total task execution delay and energy consumption, represented by the objective function Z_i, which is a function of x_{ij}, f_i, p_{ij}, and f_i^j. Since x_{ij} is a discrete variable, and p_{ij}, f_i, and f_i^j are continuous variables with highly nonlinear functional relationships, the problem exhibits non-convexity. This non-continuity undermines the convexity of the optimization problem. By relaxing x_{ij} to a continuous variable within the range $[0, 1]$, Z_i becomes a linear function of x_{ij}. Since linear functions are inherently convex, the objective function is convex with respect to x_{ij}.

The portion of the objective function related to f_i is expressed as $k(f_i)^3 T_i$, which is typically a linear function of f_i. Consequently, $k(f_i)^3 T_i$ is a convex function with respect to f_i. As convex functions remain convex under positive linear combinations, the objective function is convex with respect to f_i. The term related to p_{ij} appears in the transmission delay expression, where the transmission delay is defined as $T_i^{tr} = \frac{D_i}{R_{ij}}$, and the communication rate is given by $R_{ij} = B \log_2(1 + \frac{p_{ij} g_{ij}}{\sigma^2})$. The communication rate R_{ij} is a concave function of p_{ij}, and its reciprocal $\frac{1}{R_{ij}}$ is a convex function. Therefore, the transmission delay T_i^{tr} is convex with respect to p_{ij}. Additionally, the energy consumption term $P_i T_i^{tr}$ is a linear function of T_i^{tr}, which inherits its convexity. Thus, the objective function is convex with respect to p_{ij}.

Regarding f_i^j, the objective function involves the task execution delay, defined as $T_i^j = \frac{L_i}{f_i^j}$, which is the reciprocal of f_i^j. By computing its derivatives, it can be shown that the second derivative of T_i^j is always greater than zero, confirming that T_i^j is convex with respect to f_i^j. Furthermore, the constraint C5:$\sum_{i \in N_i} x_{ij} f_i^j \leq f_j$, which involves f_i^j, is linear and hence also convex.

Overall, both the objective function and constraints are convex, making the optimization problem convex. It can be efficiently solved using standard convex methods. Since it satisfies

Authorized licensed use limited to: Goa University. Downloaded on October 26,2026 at 08:20:24 UTC from IEEE Xplore. Restrictions apply.

the Slater condition, the problem can be solved with the Lagrangian dual method, ensuring strong duality and that the dual solution equals the optimal solution.

REFERENCES

[1] C. Zhang et al., "Implementation of a V2P-based VRU warning system with C-V2X technology," *IEEE Access*, vol. 11, pp. 69903–69915, 2023.

[2] Y. Xia, H. Zhang, X. Zhou, and D. Yuan, "Location-aware and delay-minimizing task offloading in vehicular edge computing networks," *IEEE Trans. Veh. Technol.*, vol. 72, no. 12, pp. 16266–16279, Dec. 2023.

[3] O. K. Tonguz and R. Zhang, "Harnessing vehicular broadcast communications: DSRC-actuated traffic control," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 2, pp. 509–520, Feb. 2020.

[4] M. Gao et al., "Computation offloading with instantaneous load billing for mobile edge computing," *IEEE Trans. Serv. Comput.*, vol. 15, no. 3, pp. 1473–1485, May/Jun. 2022.

[5] R. Li, C. S. Lim, M. E. Rana, and X. Zhou, "A trade-off task-offloading scheme in multi-user multi-task mobile edge computing," *IEEE Access*, vol. 10, pp. 129884–129898, 2022.

[6] Q. Shen, B.-J. Hu, and E. Xia, "Dependency-aware task offloading and service caching in vehicular edge computing," *IEEE Trans. Veh. Technol.*, vol. 71, no. 12, pp. 13182–13197, Dec. 2022.

[7] L. Song, G. Sun, H. Yu, and M. Guizani, "SD-AETO: Service-deployment-enabled adaptive edge task offloading scheme in MEC," *IEEE Internet Things J.*, vol. 10, no. 21, pp. 19296–19311, Nov. 2023.

[8] H. Xu et al., "Multiuser computation offloading for long-term sequential tasks in mobile edge computing environments," *Tsinghua Sci. Technol.*, vol. 28, no. 1, pp. 93–104, Feb. 2023.

[9] D. Alsadie, "Efficient task offloading strategy for energy-constrained edge computing environments: A hybrid optimization approach," *IEEE Access*, vol. 12, pp. 85089–85102, 2024.

[10] A. Mahjoubi, A. Ramaswamy, and K.-J. Grinnemo, "An online simulated annealing-based task offloading strategy for a mobile edge architecture," *IEEE Access*, vol. 12, pp. 70707–70718, 2024.

[11] L. Tang et al., "Joint optimization of network selection and task offloading for vehicular edge computing," *J. Cloud Comput.*, vol. 10, 2021, Art. no. 23.

[12] T. Cui et al., "Task offloading based on Lyapunov optimization for MEC-assisted vehicular platooning networks," *Sensors*, vol. 19, no. 22, 2019, Art. no. 4974.

[13] Y. Li, Z. Liu, and Q. Tao, "A resource allocation strategy for Internet of Vehicles using reinforcement learning in edge computing environment," *Soft Comput.*, vol. 27, pp. 3999–4009, 2023.

[14] Z. Zhang and F. Zeng, "Efficient task allocation for computation offloading in vehicular edge computing," *IEEE Internet Things J.*, vol. 10, no. 6, pp. 5595–5606, Mar. 2023.

[15] H. Yang, Z. Wei, Z. Feng, X. Chen, Y. Li, and P. Zhang, "Intelligent computation offloading for MEC-based cooperative vehicle infrastructure system: A deep reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 71, no. 7, pp. 7665–7679, Jul. 2022.

[16] J. Su, Z. Liu, Y. -a. Xie, Y. Li, K. Ma, and X. Guan, "UEE-delay balanced online resource optimization for cooperative MEC-enabled task offloading in dynamic vehicular networks," *IEEE Internet Things J.*, vol. 11, no. 7, pp. 11496–11507, Apr. 2024.

[17] Z. Zhang, Z. Chen, Y. Shen, X. Dong, and N. Xi, "A dynamic task offloading scheme based on location forecasting for Mobile Intelligent Vehicles," *IEEE Trans. Veh. Technol.*, vol. 73, no. 6, pp. 7532–7546, Jun. 2024.

[18] X. Peng et al., "Deep reinforcement learning for shared offloading strategy in Vehicle Edge Computing," *IEEE Syst. J.*, vol. 17, no. 2, pp. 2089–2100, Jun. 2023.

[19] T. Zhang, F. Wu, Z. Chen, and S. Chen, "Optimization of edge–cloud collaborative computing resource management for Internet of Vehicles based on multiagent deep reinforcement learning," *IEEE Internet Things J.*, vol. 11, no. 22, pp. 36114–36126, Nov. 2024.

[20] D. B. Son, H. K. Vo, T. H. Binh, T. H. Hai, B. M. Nguyen, and H. T. T. Binh, "Reinforcement-learning-based deadline constrained task offloading schema for energy saving in vehicular edge computing system," in *Proc. Int. Joint Conf. Neural Netw.*, Gold Coast, QLD, Australia, 2023, pp. 1–8.

[21] K. Mishra, G. N. V. Rajareddy, U. Ghugar, G. S. Chhabra, and A. H. Gandomi, "A collaborative computation and offloading for compute-intensive and latency-sensitive dependency-aware tasks in dew-enabled vehicular fog computing: A federated deep q-learning approach," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 4, pp. 4600–4614, Dec. 2023.

[22] G. Ma et al., "Joint partial offloading and resource allocation for vehicular federated learning tasks," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 8, pp. 8444–8459, Aug. 2024.

[23] R. Duo et al., "SDN-based handover scheme in cellular/IEEE 802.11p hybrid vehicular networks," *Sensors*, vol. 20, no. 4, 2020, Art. no. 1082.

[24] I. Labriji et al., "Mobility aware and dynamic migration of MEC services for the Internet of Vehicles," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 1, pp. 570–584, Mar. 2021.

[25] Z. Fernández et al., "Challenges and solutions for service continuity in inter-PLMN handover for vehicular applications," *IEEE Access*, vol. 11, pp. 8904–8919, 2023.

[26] J. Du et al., "Vehicle-road cooperative task offloading with task migration in MEC-enabled IoV," in *Proc. 17th Int. Conf. Wireless Algorithms, Syst.*, 2022, pp. 261–272.

[27] C. Chen, Y. Zeng, H. Li, Y. Liu, and S. Wan, "A multihop task offloading decision model in MEC-enabled Internet of Vehicles," *IEEE Internet Things J.*, vol. 10, no. 4, pp. 3215–3230, Feb. 2023.

[28] Z. Liao, S. Xu, J. Huang, and J. Wang, "Task migration and resource allocation scheme in IoV with roadside unit," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 4, pp. 4528–4541, Dec. 2023.

[29] L. Zhao et al., "MESON: A mobility-aware dependent task offloading scheme for urban vehicular edge computing," *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 4259–4272, May 2024.

[30] S. Zhang, N. Yi, and Y. Ma, "A survey of computation offloading with task types," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 8, pp. 8313–8333, Aug. 2024.

[31] J. Gong, Y. Yang, Z. Wang, D. Wang, X. Liu, and M. Peng, "Frequency-Doppler division multiplexing modulation—A waveform design for high Doppler communications," *IEEE Commun. Lett.*, vol. 28, no. 6, pp. 1288–1292, Jun. 2024.

[32] X. Fan, W. Gu, C. Long, C. Gu, and S. He, "Optimizing task offloading and resource allocation in vehicular edge computing based on heterogeneous cellular networks," *IEEE Trans. Veh. Technol.*, vol. 73, no. 5, pp. 7175–7187, May 2024.

[33] P. Yue, W. Yue, P. Duan, Y. Fan, and C. Li, "CAVs as a mobile computing platform: Task offloading strategy in mixed traffic systems," *IEEE Internet Things J.*, vol. 11, no. 20, pp. 33592–33603, Oct. 2024.

[34] B. Shi and Y. Wu, "Task offloading and resource allocation strategies among multiple edge servers," *IEEE Internet Things J.*, vol. 11, no. 8, pp. 14647–14656, Apr. 2024.

[35] K. Tan, L. Feng, G. Dán, and M. Törngren, "Decentralized convex optimization for joint task offloading and resource allocation of vehicular edge computing systems," *IEEE Trans. Veh. Technol.*, vol. 71, no. 12, pp. 13226–13241, Dec. 2022.

[36] N. Abbas, W. Fawaz, S. Sharafeddine, A. Mourad, and C. Abou-Rjeily, "SVM-based task admission control and computation offloading using Lyapunov optimization in heterogeneous MEC network," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 3, pp. 3121–3135, Sep.. 2022.

[37] M. P. John Mahenge, C. Li, and C. A. Sanga, "Energy-efficient task offloading strategy in mobile edge computing for resource-intensive mobile applications," *Digit. Commun. Netw.*, vol. 8, no. 6, pp. 1048–1058, 2022.

[38] J. Zhang, H. Guo, J. Liu, and Y. Zhang, "Task offloading in vehicular edge computing networks: A load-balancing solution," *IEEE Trans. Veh. Technol.*, vol. 69, no. 2, pp. 2092–2104, Feb. 2020.

[39] A. Naouri, H. Wu, N. A. Nouri, S. Dhelim, and H. Ning, "A novel framework for mobile-edge computing by optimizing task offloading," *IEEE Internet Things J.*, vol. 8, no. 16, pp. 13065–13076, Aug. 2021.



Li Jin received the B.S. degree in computer science and technology degree from the Nanjing University of Aeronautics and Astronautic, Nanjing City, China, in 2004, the M.S. degree in soft engineering from Southeast University, Nanjing City, China in 2007, the Ph.D. degree in communication and information system from Nantong University, Nantong City, China, in 2020. She is currently an Associate Professor with the School of Information Science and Technology, Nantong University. Her research interests include wireless communication, artificial intelligence, and vehicular networks.

Authorized licensed use limited to: Goa University. Downloaded on October 26,2026 at 08:20:24 UTC from IEEE Xplore. Restrictions apply.



Shuaili Bao received the B.E. degree in information management and information system from the Jiangsu University of Technology, Changzhou, China, in 2023. She is currently working toward the master's degree with the School of Artificial Intelligence and Computer Science, Nantong University, Nantong, China. Her research focuses on edge computing in vehicular networks.



Guoan Zhang (Senior Member, IEEE) received the B.S. degree in precision instruments, the M.S. degree in automatic instruments and equipment, and the Ph.D. degree in communication and information systems from Southeast University, Nanjing, China, in 1986, 1989, and 2001, respectively. He is currently a Full Professor with the School of Electronics and Information, Nantong University, Nantong, China. His research interests include cognitive wireless networks and vehicular ad hoc networks.



Hao Zhu received M.S. degree from Jiang Su University, Zhen jiang City, China, in 2004, and the Ph. D. degree from the Nanjing University of Aeronautics and Astronautics, Nanjing City, China. He is currently an Associate Professor with the School of Artificial Intelligence and Computer Science, Nantong University, Nantong, China. His research interests include network security, artificial intelligence and vehicular networks.



Wei Duan received the Ph.D. degree from Chonbuk National University, Jeonju, South Korea, in 2017. He is currently a Full Professor with Nantong University, Nantong, China. He has participated with World Class University Project, sponsored by the National Research Foundation of Korea Grant funded by the Korean Ministry of Education Science and Technology, as a Vice Head Researcher. He has authored more than 70 journal articles. His research interests include a variety of topics in the areas of wireless communications. He is also a Guest Editor of IEEE INTERNET OF THINGS Special Issue on Near-Field Communications (NFC) in Internet of Everything, IEEE NETWORK Special Issue on Near-Field Communications: Challenges and Opportunities in the Networking Landscape, and *China Communications* Special Issue on New Advances in Nonorthogonal Multiple Access; and a Lead Guest Editor for *Wireless Communications and Mobile Computing* Special Issue on Architectures, Challenges, and Opportunities within 6G Emerging Technologies. He is an Editor of *Frontiers in Communications and Networks*.



Qiang Sun (Member, IEEE) received the Ph.D. degree in communications and information systems from Southeast University, Nanjing, China, in 2014. In 2016, he was a Visiting Scholar with the University of Delaware, Newark, DE, USA. He is currently a Professor with the School of Information Science and Technology, Nantong, China. His research interests include deep learning and wireless communications.



Jiayi Zhang (Senior Member, IEEE) received the B.Sc. and Ph.D. degrees in communication engineering from Beijing Jiaotong University, Beijing, China, in 2007 and 2014, respectively. Since 2016, he has been a Professor with the School of Electronic and Information Engineering, Beijing Jiaotong University. From 2012 to 2013, he was a Visiting Scholar with the Wireless Group, University of Southampton, U.K. From 2014 to 2016, he was a Postdoctoral Research Associate with the Department of Electronic Engineering, Tsinghua University, China. From 2014 to 2015, he was also a Humboldt Research Fellow with the Institute for Digital Communications, Friedrich-Alexander-University Erlangen-Nürnberg (FAU), Germany. His research interests include massive MIMO, reconfigurable intelligent surfaces, communication theory, and applied mathematics. He was the recipient of the Best Paper awards at *WCSP 2017* and *IEEE APCC 2017*. He was recognized as an Exemplary Reviewer of IEEE COMMUNICATIONS LETTERS in 2015 and 2016, and of IEEE TRANSACTIONS ON COMMUNICATIONS from 2017 to 2019. He was the Lead Guest Editor of the special issue on "Multiple Antenna Technologies for Beyond 5G" in the IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS. He is also an Associate Editor for IEEE TRANSACTIONS ON COMMUNICATIONS, IEEE COMMUNICATIONS LETTERS, IEEE ACCESS, and *IET Communications*.



Pin-Han Ho (Fellow, IEEE) received the Ph.D. degree from Queens University, Kingston, ON, Canada, in 2002. He is currently a Full Professor with the Department of Electrical and Computer Engineering, University of Waterloo. He is the author/coauthor of more than 400 refereed technical papers, several book chapters, and the coauthor of two books on Internet and optical network survivability. His research interests include wide range of topics in broadband wired and wireless communication networks, including wireless transmission techniques, mobile system design and optimization, and network dimensioning and resource allocation. He is in the rank of IEEE Fellow and a Professional Engineer Ontario (PEO).

Authorized licensed use limited to: Goa University. Downloaded on October 26,2026 at 08:20:24 UTC from IEEE Xplore. Restrictions apply.